

[B] TPMOS_TEXTBOOK [GRAD]

The Definitive Guide to the Mono-OS ()

Version 6.00 (Standardized Ops Edition)

Softness wins. The empty center of the flexbox holds ten thousand things. [ZEN 禪]

[DOC] Table of Contents ()

[DNA] Part I: The Philosophy of Pieces

- CH1: The Soul of a Piece (Standardized Ops Edition)
- CH2: The File System as the Source of Truth
- CH3: The Standardized Pipeline
- CH4: Development & The Standardized Ops Flow
- CH5: System Apps & The Centralized Core

Part II: The Developer's Tools

- CH6: PAL: The Assembly Language of TPMOS
- CH7: Fuzz-Op & Op-Ed: The Developer's Forge
- CH8: GL-OS: The Transition to High-Fidelity
- CH9: Automated Testing of the Ops Pipeline
- CH10: Future Horizons: AI, P2P, and Finance

[REC] Part III: Advanced Piece Concepts

- CH11: The Recursive Forge: Projects Building Projects
- CH12: The Simulation Theater: CHTPM & View.txt
- CH13: Business Strategy: The Economics of Pieces
- CH14: The Soul Pen: Lore & AI Creativity
- CH15: Cross-Platform: Mac, Linux, and Windows

[TOOL] Part IV: Debugging & Sovereignty

- CH16: Pitfalls & Debugging the Ops Pipeline
- CH17: Exo-Sovereignty: The Independent Node
- CH18: Dynamic Trait Menus: UI from DNA

[COSMOS] Part V: The Future Expansion (NEW)

- CH19: THE THEATER: CHTPM Markup & Parser Logic
- CH20: THE OPENGL SHELL: CHTMGL & High-Fidelity
- CH21: P2P-NET & TSOTS-ONLINE: Elegant Decentralization
- CH22: THE AI BRAIN: LLM & API Interfaces
- CH23: THE FINANCIAL FORGE: Real-Time Economics
- CH24: THE PROJECT CATALOG: Academic Summaries
- CH25: MARKETING & COMMUNITY: The Vibe Ecosystem
- GLOSSARY.md

- QUIZ.md
- ANSWER_KEY.md

[DNA] Chapter 1: The Soul of a Piece (Standardized Ops Edition)

The True Piece Method (TPM) is the governing philosophy of this entire OS. In TPMOS v6.00, we have moved beyond simple object management into Standardized Ops Architecture. [PIECE 砖]

[BLD] The PMO Hierarchy

The system follows a strict thought priority. If you violate this, the "Mirror" will crack! [MIRROR 镜]

1. PIECE (The Atomic Unit / Soul) [PIECE 砖]

- Everything is a Piece. A button, a player, a map, or a galaxy.
- **Rule:** A Piece owns its state **exclusively**.
- **File:** `piece.pdl` (DNA) + `state.txt` (Mirror).

2. MODULE (The Logic Agent / Brain) [MIND 脑]

- The Module is the "Manager". In v6.00, we mandate **Thin Brain** design.
- **Mandate:** The Module polls input and delegates logic to **Ops**. It does not perform heavy computation itself.

3. OS / CHTPM (The Theater / View) [VIEW 戏]

- The OS is the stage where Pieces perform. It handles the UI, the layouts, and the "Magic" (variable substitution).
 - **Standardized Ops:** Every interaction is now routed through a centralized ops registry.
-

[DNA] Anatomy of Piece DNA (`pdl`)

Every Piece has a `pdl` file. This is its blueprint.

```
<piece_id>my_hero</piece_id>
<traits>
  <trait>movable</trait>
  <trait>auditable</trait>
</traits>
<methods>
  <method id="move_north" cmd="user::move_entity north" />
  <method id="say_hello" cmd="echo 'Hello' > last_response.txt" />
</methods>
```

[MIRROR] The Mirror (`state.txt`)

While DNA defines the **potential**, the Mirror defines the **now**. It is a flat text file for high-speed reading.

- `x=10`
- `y=5`
- `health=100`
- `status=idle`

[TIPS] **Pro Tip:** "If it's not in a file, it's a lie." We never trust memory. In the Standardized Ops Edition, even

transient animation states are mirrored to ensure total auditability.

Fondu: The Project Lifecycle

Version 6.00 introduces Fondu, the project lifecycle manager. Projects are no longer loosely scattered; they are installed, registered, and managed through a centralized system.

- ``fondu --install <project>``: Compiles the source, deploys to ``pieces/apps/installed/``, and registers available ops.
 - ``fondu --list-ops``: Shows every capability available to the system, from movement to AI inference.
-

[ZEN] The Zen of TPM

- ****Data Sovereignty:**** No piece touches another's files without permission.
 - ****Auditability:**** Every major change is logged to the ``master_ledger.txt``.
 - ****Recursive Reality:**** A Piece can contain other Pieces. This is the ****Scale-Free Container Model****.
-

[BLD] Scholar's Corner: The "Mirror of Tomokazu"

There is a legendary anecdote among TPMOS engineers known as the "Mirror of Tomokazu." Early in development, a rogue developer tried to optimize the system by keeping a player's HP in a global variable instead of writing it to ``state.txt``. During a stress test, the process crashed. When the system rebooted, the player had 0 HP and was "permanently dead," despite having been full health seconds before. This disaster led to the founding mandate: "If it's not in a file, it's a lie." From that day on, every piece of reality had to be mirrored in a file, ensuring immortality through persistence. [IMM 燭]

[DOC] Study Questions

1. What are the three layers of the PMO hierarchy in v6.00?
 2. What is the role of Fondu in the Standardized Ops architecture?
 3. Why is a "Thin Brain" preferred over a "Heavy Brain" in module design?
 4. True or False: A galaxy and a button can both be considered "Pieces" in TPMOS.
-

[\[Return to Index\]\(INDEX.md\)](#)

Chapter 2: The File System as the Source of Truth

In TPMOS, the file system isn't just storage; it is the active state of the universe. We do not use databases; we use directories.

[BLD] The Standardized Directory Structure

Every project follows a strict anatomy to be compatible with Fondu.

```

projects/my_app/
├── project.pdl      # Project metadata & entry point
├── layouts/        # CHTPM Markup files (The View)
├── manager/        # C/PAL Brain (The Controller)
├── ops/            # C binaries (The Muscle)
│   ├── ops_manifest.txt # List of exposed capabilities
│   └── +x/         # Compiled binaries
├── pieces/         # The Souls (The Data)
│   ├── player_01/
│   │   ├── piece.pdl
│   │   └── state.txt
│   └── assets/     # Icons, Artifacts, Audio

```

The Ops Registry (`pieces/os/ops\_registry/`)

When a project is installed via `fondu`, its ops are registered in a central registry. This allows any project to call another project's "Muscles".

Example `ops\_registry/user.txt`:

```

user::create_profile=projects/user/ops/+x/create_profile.+x
user::move_entity=projects/user/ops/+x/move_entity.+x

```

[MIND] The Thin Brain Pattern

A Standardized Ops Module (`manager.c`) follows this logic:

1. Poll: Read `history.txt` or `gui\_state.txt`.
2. Match: See if the user clicked a button or typed a command.
3. Execute: Look up the op in the registry and fork a process.
4. Reset: Clear the input buffer.

Data Sovereignty & The Mirror

Each Piece has a folder. Inside that folder, `state.txt` is the Mirror.

- **Atomic Updates:** We use `rename()` to ensure state updates are atomic. Write to `state.txt.tmp`, then swap.
- **Key-Value Pairs:** State is always `key=value`. No binary blobs.

Master Ledger

Every state-changing Op MUST log to the global `master\_ledger.txt`. This allows for "Time Travel Debugging" - you can literally replay the ledger to see how the system reached a specific state.

Code Example: Reading State in C

The following utility is used across all "Thin Brain" managers to read state efficiently.

```
char* get_state_value(const char* piece_path, const char* key) {
    char state_path[512];
    snprintf(state_path, sizeof(state_path), "%s/state.txt", piece_path);
    FILE* f = fopen(state_path, "r");
    if (!f) return NULL;

    static char value[256];
    char line[512];
    while (fgets(line, sizeof(line), f)) {
        char* eq = strchr(line, '=');
        if (eq) {
            *eq = '\0';
            if (strcmp(line, key) == 0) {
                char* val = eq + 1;
                val[strcspn(val, "\n\r")] = 0;
                strncpy(value, val, 255);
                fclose(f);
                return value;
            }
        }
    }
    fclose(f);
    return NULL;
}
```

[BLD] Scholar's Corner: The "Directory Ghost"

There's a famous bug report from v4.2 where a developer deleted a folder but the "Piece" kept appearing in the UI. It turned out the Renderer was caching directory listings for 5 seconds to save battery. This led to the concept of the "Directory Ghost" - a piece of data that exists in the "Theater" but not in "Reality." In v6.00, we've implemented State-First Polling, ensuring the Theater only renders what the File System confirms exists. [GHOST 鬼]

[DOC] Study Questions

1. What is the `ops\_manifest.txt` and why is it important for Fondu?
2. Explain the "Thin Brain" pattern.
3. Why does TPMOS use `rename()` for state updates?
4. How does a centralized `ops\_registry` enable cross-project collaboration?

[\[Return to Index\]\(INDEX.md\)](#)

[LAUNCH] Chapter 3: The Standardized Pipeline

In TPMOS v6.00, the pipeline from user input to visual feedback is more structured than ever. This is the Standardized Ops Pipeline. [TOOL 齿]

The Feedback Loop

The loop consists of four distinct phases, all mediated by the file system.

1. INPUT (The Sensation) [CPU 键]
 - User types in the CLI or clicks a CHTPM button.
 - Input is written to `pieces/keyboard/history.txt` or `pieces/apps/gl\_os/session/gui\_state.txt`.
2. BRAIN (The Module) [MIND 脑]
 - A background manager (e.g., `playrm\_manager.+x`) polls the input files.
 - It identifies a command (e.g., `MOVE:up`).
 - It looks up the corresponding ****Op**** in the Standardized Registry.
3. MUSCLE (The Op)
 - The manager forks a process to run the Op binary (e.g., `move\_entity.+x`).
 - The Op modifies the ****Mirror**** (`state.txt`) of the target Piece.
 - The Op updates the `frame\_changed.txt` pulse.
4. THEATER (The View) [VIEW 戏]
 - The `chtpm\_parser` detects the pulse.
 - It re-reads the layouts and substitutes variables (e.g., `\${player\_x}`).
 - It composes the final ASCII or GL frame.

Fondu and the Registry

The Standardized Pipeline relies on every project being correctly registered.

The Lifecycle of a Pulse

When an Op finishes its work, it doesn't just exit. it must signal the OS that "Reality has shifted."

```
// pulse.c - Standard Op Signal
void trigger_pulse() {
    FILE* f = fopen("pieces/display/frame_changed.txt", "w");
    if (f) {
        fprintf(f, "%ld", (long)time(NULL));
        fclose(f);
    }
}
```

[MIND] Thin Brain vs. Heavy Brain

- **Legacy (Heavy Brain):** The manager did everything. It read state, calculated physics, updated files, and rendered. This led to "Sync Lag."
 - **Standardized (Thin Brain):** The manager only routes.
 - **Input:** "I want to jump."
 - **Manager:** "Okay, I will tell the `jump` muscle to handle that."
 - **Manager:** (Forks `jump.+x` and goes back to listening).
-

Code Example: The Thin Brain Loop

```
while (keep_running) {
    if (check_history_changed()) {
        char* last_cmd = get_last_history_line();
        if (strstr(last_cmd, "MOVE:")) {
            char* direction = last_cmd + 5;
            execute_op("user::move_entity", direction);
        }
        clear_history();
    }
    usleep(10000); // 100Hz polling = CPU Friendly
}
```

[BLD] Scholar's Corner: The "Pulse That Broke the World"

In the early days of the Standardized Pipeline, a developer accidentally wrote a "Infinite Pulse." An Op would trigger a pulse, which triggered a Module, which triggered the same Op... creating a feedback loop that froze the entire OS and heated up the CPU to 90°C. This incident led to the Pulse Cooldown Rule: "No piece may trigger more than 60 pulses per second."

[DOC] Study Questions

1. Describe the four phases of the Standardized Ops Pipeline.
 2. How does `frame\_changed.txt` facilitate communication between the Muscle and the Theater?
 3. Why is the "Thin Brain" model more stable than the "Heavy Brain" model?
 4. Scenario: You click a button, but nothing happens. Which phase of the pipeline is likely failing?
-

[\[Return to Index\]\(INDEX.md\)](#)

[TOOL] Chapter 4: Development & The Standardized Ops Flow

Developing for TPMOS v6.00 requires a shift in mindset. You aren't building an "app"; you are building a Collection of Ops managed by a Thin Brain. [BLD 建]

[LAUNCH] The 5-Step Project Flow

Follow this sequence to build a project that is 100% compliant with the Standardized Ops Edition.

1. Define the DNA (`project.pdl`)

Tell the OS what your project is and where its heart lies.

SECTION	KEY	VALUE
META	project_id	my_game
META	entry_layout	projects/my_game/layouts/main.chtpm

2. Create the View (`main.chtpm`)

Design your UI using CHTPM markup. Use `\${vars}` to bind to future state.

```
<layout id="my_game">
  <text label="Score: ${player_score}" />
  <button label="Jump" onClick="OP:my_game::jump" />
</layout>
```

3. Build the Muscles (`ops/\*.c`)

Each capability (Jump, Shoot, Save) should be a standalone C file.

```
// jump.c
int main() {
  int current_y = read_state("pieces/player/state.txt", "y");
  write_state("pieces/player/state.txt", "y", current_y + 1);
  trigger_pulse();
  return 0;
}
```

4. Register the Ops (`ops\_manifest.txt`)

List your muscles so Fondu can register them.

```
my_game::jump=projects/my_game/ops/+x/jump.+x
```

5. Install via Fondu

Run the installer to wire everything into the OS.

```
./fondu --install my_game
```

[TOOL] The Developer's Toolkit

TPMOS provides several built-in tools to speed up this flow:

- `op-ed`: A visual editor for creating Pieces and binding Ops to methods.
- `fuzz-op`: A testing suite that hammers your Ops with random data to ensure they don't crash.
- `man-ops`: A manual entry generator for your project.

Code Example: A Standardized Makefile

In v6.00, your Makefile should always output to the `+x/` directory within your project folder.

```
CC=gcc
CFLAGS=-O2 -Wall -Wextra
TARGETS=ops/+x/jump.+x ops/+x/shoot.+x

all: $(TARGETS)

ops/+x/%.+x: ops/%.c
    @mkdir -p ops/+x
    $(CC) $(CFLAGS) $< -o $@

clean:
    rm -rf ops/+x
```

[BLD] Scholar's Corner: The "Standardization Riot"

When Fondu and the Standardized Ops mandate were first introduced, there was a minor "riot" among developers who preferred the freedom of the legacy v5.01 system. They argued that creating separate binaries for every action was "too much work." However, the riot ended when they realized that Standardized Ops allowed them to share code instantly. A "Jump" muscle written for a platformer could be reused in an RPG with zero changes. Efficiency won the day! [BLD 馆][MKT↑ 升]

[DOC] Study Questions

1. What are the 5 steps to building a Standardized Ops project?
2. Why is it important to use `trigger_pulse()` at the end of an Op?
3. What is the benefit of using `fondu --install` over manually copying files?
4. Exercise: Design a `project.pdl` for a simple "Calculator" app.

[Return to Index](INDEX.md)

[CPU] Chapter 5: System Apps & The Centralized Core

TPMOS v6.00 is built on a foundation of Standardized System Apps. These are the core utilities that manage the OS itself. [BLD 馆]

[BLD] The Core Quartet

Every TPMOS session relies on these four pillars.

1. The Process Manager (``pieces/os/process_manager``)

The invisible hand that ensures Modules are running.

- **Standardized Behavior:** It reads ``compiled_projects.txt`` and ensures every installed app's manager is active.
- **Safety:** It monitors CPU usage and kills runaway "Heavy Brains."

2. Fondu (``pieces/system/fondu``)

The Lifecycle Authority.

- Handles ``--install``, ``--uninstall``, and ``--archive``.
- Maintains the **Global Ops Catalog**.

3. The Playroom (``projects/playrm``)

The primary game engine and world simulator.

- Manages 2D ASCII maps and Entity positions.
- Uses ``move_entity`` and ``render_map`` ops.

4. Op-Ed (``projects/op-ed``)

The Integrated Development Environment (IDE).

- Allows for real-time manipulation of Piece state.
- Includes the **PAL Editor** for visual scripting.

The Global Ops Catalog

Located at ``pieces/os/ops_catalog.txt``, this file is the "Phonebook" of the OS. Any process can read it to find out what capabilities are available on the system.

Example ``ops_catalog.txt``:

```
system::shutdown
user::login
playrm::spawn_npc
ai::inference
```

Code Example: Calling a System Op from a Module

```
void shutdown_system() {  
    char* op_path = lookup_op("system::shutdown");  
    if (op_path) {  
        system(op_path);  
    }  
}
```

[BLD] Scholar's Corner: The "App Store of 2026"

In early 2026, a developer used Fondu to create a "Project Pack" that contained 50 small games. Because they all followed Standardized Ops, they all worked together instantly. A player could use their character from Game 1 to enter a shop in Game 2 and buy an item from Game 3. This became known as the "Sovereign App Store" - a marketplace of interoperable pieces rather than siloed applications.

[DOC] Study Questions

1. Name the "Core Quartet" of TPMOS system apps.
2. What is the `ops\_catalog.txt` and how does it benefit developers?
3. How does the Process Manager ensure system stability?
4. Critical Thinking: Why is the Playroom considered a "System App" even though it looks like a game engine?

[\[Return to Index\]\(INDEX.md\)](#)

Chapter 6: PAL: The Assembly Language of TPMOS

How do you orchestrate complex operations across an entire project? You write a PAL script. PAL (Prisc Assembly Language) is the glue that holds the Standardized Ops architecture together. [LINK 接][MIND 脑]

What is PAL?

PAL is an assembly-style scripting language designed specifically for TPMOS. It lets you chain together Ops (Muscles) into complex workflows without writing new C code for every variation.

The Instruction Set (v6.00)

PAL has a RISC-inspired instruction set, now fully aware of the Standardized Registry:

Instruction	Purpose	Example
-----	-----	-----
`OP`	Call a standardized op	`OP user::move_entity "player" "up"`
`call`	Call a local subroutine	`call game_loop`
`sleep`	Pause execution (ms)	`sleep 100`
`beq`	Branch if equal	`beq r0, r1, done`
`read_state`	Read piece state	`read_state r1, "player", "pos_x"`
`hit_frame`	Trigger render pulse	`hit_frame`

PAL + Fondu Integration

In the Standardized Ops Edition, PAL scripts are considered First-Class Citizens. When you install a project via Fondu, its scripts are registered alongside its binaries.

1. Ops: Compiled C binaries for core logic.
2. Scripts: PAL `.asm` files for high-level orchestration.

Now any project can call a script as if it were a binary:

```
./fondu --install user
# Now call a script registered in the manifest
user::welcome_sequence.asm "new_player"
```

Code Example: The PAL Interpreter Logic

The interpreter (`prisc+x.c`) handles the `OP` instruction by looking up the path in the `ops\_catalog.txt`.

```
case OP_CUSTOM: {
    // Standardized Op Lookup
    char* op_path = lookup_standardized_op(inst->custom_name);
    if (op_path) {
```

```

char cmd[4096];
snprintf(cmd, sizeof(cmd), "%s %s %s", op_path, arg1, arg2);
run_command(cmd);
}
break;
}

```

The PAL Editor (GUI Scripting)

Within `op-ed`, developers can now use a visual palette to build PAL scripts. This bridges the gap between raw code and high-level logic.

[MAP] The Scratch/RPGMaker Influence

To make development accessible, we are implementing high-level abstractions:

- **Forever Loops:** For background sensing logic.
- **Broadcasts:** For cross-piece communication.
- **Variable Blocks:** For score and health management.

[DOC] PAL Program Example: Treasure Hunt

```

; treasure_hunt.asm
start:
    OP user::create_profile "hunter"
    sleep 200

; Move 5 steps right
    OP user::move_entity "hunter" "right"
    sleep 100
; ... repeat 4 more times ...

; Check state
    read_state r1, "hunter", "happiness"
    addi r2, r0, 60
    beq r1, r2, treasure_found

    OP playrm::render_map
    halt

treasure_found:
    OP playrm::fuzzpet_action "hunter" "celebrate"
    hit_frame
    halt

```

[BLD] Scholar's Corner: The "Assembly That Assembled Itself"

In the early days of TPMOS, a developer wrote a PAL script to automate the process of creating new PAL scripts. One day, the developer accidentally pointed the script at itself. Within seconds, there were 10,000 copies of the script filling the disk. This became known as "The Assembly That Assembled Itself." It taught us the critical importance of bounded loops and the power (and danger) of self-referential systems in v6.00.

[INF 无]

[DOC] Study Questions

1. How does the `OP` instruction in PAL interact with the `ops\_catalog.txt`?
2. What is the benefit of using a PAL script to orchestrate multiple C-based Ops?
3. Why is `hit\_frame` essential at the end of a PAL script?
4. True or False: PAL scripts share memory with the C module that calls them.

[\[Return to Index\]\(INDEX.md\)](#)

[LAB] Chapter 7: Fuzz-Op & Op-Ed: The Developer's Forge

In TPMOS v6.00, we don't just write code; we Forge it. This chapter covers the two primary tools for ensuring your Standardized Ops are robust and beautiful. [* 閃]

Fuzz-Op: The Chaos Engine

`fuzz-op` is a stress-testing utility that ensures your Ops are "Bulletproof." It works by identifying every Op registered in your `ops\_manifest.txt` and bombarding it with random, malicious, or edge-case data.

The Fuzzing Strategy

1. Argument Injection: Passing empty strings, 1MB strings, or special characters (;, `&`, `|`) to see if your Op crashes or leaks shell access.
2. State Corruption: Modifying `state.txt` while your Op is running to test for race conditions.
3. Pulse Flooding: Triggering `frame\_changed.txt` thousands of times per second.

[TIPS] ****Developer Rule:**** "If it can't survive a 5-minute Fuzz-Op session, it's not ready for the Mainframe."

[GL] Op-Ed: The Sovereign IDE

`op-ed` (Op Editor) is a system app that allows you to manipulate the universe in real-time.

Key Features of Op-Ed v6.00

- ****Piece Inspector:**** Click any Piece on a map to see its DNA (`.pdl`) and Mirror (`state.txt`).
- ****Live Injection:**** Manually change a value (e.g., `hp=999`) and see the Theater update instantly.
- ****Method Binding:**** Drag an Op from the catalog and drop it onto a Piece's `on\_click` method.
- ****Visual Palette:**** Use ASCII "brushes" to paint worlds directly into `map.txt`.

Code Example: Fuzz-Op Registry Check

```
void fuzz_project(const char* project_id) {
    char* ops[] = get_registered_ops(project_id);
    for (int i = 0; i < op_count; i++) {
        printf("Fuzzing Op: %s\n", ops[i]);
        for (int j = 0; j < 100; j++) {
            char* bad_data = generate_chaos_string();
            execute_op(ops[i], bad_data);
            if (system_crashed()) report_bug(ops[i], bad_data);
        }
    }
}
```

[BLD] Scholar's Corner: The "Fuzz-Op Filter"

During the v5.9 beta, a major security flaw was found where an Op used ``system()`` without sanitizing input. A Fuzz-Op test accidentally generated the string `""; rm -rf / ;""`. Thankfully, the test was running in a restricted sandbox! This incident led to the "Fuzz-Op Filter"—a mandatory middleware that sanitizes all arguments before they reach the Op's ``main()`` function. Safety first! [SEC 安]☹

[DOC] Study Questions

1. What is the primary goal of ``fuzz-op``?
 2. How does ``op-ed`` allow for "Real-Time Reality Manipulation"?
 3. Describe the relationship between a Piece's "DNA" and its "Mirror" as seen in Op-Ed.
 4. Exercise: Design a test case for a ``withdraw_money`` Op that would catch a negative balance bug.
-

[Return to Index](INDEX.md)

Chapter 8: GL-OS: The Transition to High-Fidelity

While TPMOS was born in the world of ASCII, version 6.00 marks the beginning of the High-Fidelity Era. GL-OS is the bridge between the text terminal and the 3D GPU. [CPU 算][COSMOS 宇]

[VIEW] The Dual-Mode Theater

The CHTPM Parser (Chapter 19) is now designed to output to two different targets:

1. ASCII Target: Renders to the terminal using standard characters.
2. OpenGL Target (CHTMGL): Renders to a 3D window using Artifacts and Textures.

[BLD] What are Artifacts?

In the OpenGL Shell, we don't have "characters"; we have Artifacts.

- **Voxel Masks:** 8x8x8 bitmasks that define a 3D shape (e.g., a "Sword" or a "Tree").
- **Extrusion:** A technique where a 2D ASCII character is "pushed" into the 3D Z-axis to create depth.

[MIND] The GL-OS Module Design

A GL-OS module works identically to a standard module, but it listens for different input events:

- **Mouse Clicks (X,Y):** Converted from screen coordinates to Piece-relative coordinates.
- **Camera Swipes:** Used to rotate the 3D viewport.

Standardized Ops in GL

Ops remain identical! If you have a `move\_entity` op, it doesn't care if it's being rendered in ASCII or 3D. It simply updates `state.txt`. The Theater (GL-OS) handles the visual translation.

Code Example: Artifact Loading (Simplified)

```
void load_artifact(const char* name) {
    char path[256];
    snprintf(path, sizeof(path), "assets/artifacts/%s.txt", name);
    // Read 8x8x8 bitmask from file
    unsigned char mask[8][8];
    read_bitmask_file(path, mask);
    // Upload to GPU as a 3D Texture or Voxel Mesh
    upload_to_gpu(mask);
}
```

[BLD] Scholar's Corner: The "First Dimension Shift"

The first time a developer toggled a TPMOS project from ASCII to GL, they saw their 2D character suddenly "stand up" into 3D. It was a simple "extrude" effect, but it changed the project forever. This moment, known

as the "First Dimension Shift," proved that the Piece Method was truly scale-free—logic doesn't care about the number of dimensions it exists in. [* 閃]

[DOC] Study Questions

1. What is an "Artifact" and how does it differ from an ASCII character?
 2. How does GL-OS maintain compatibility with existing Standardized Ops?
 3. Explain the concept of "Extrusion" in the context of CHTMGL.
 4. Critical Thinking: Why is it important to keep the "Brain" (Module) separate from the "Theater" (GL-OS)?
-

[Return to Index](INDEX.md)

[LAB] Chapter 9: Automated Testing of the Ops Pipeline

In a system where every action is a standalone binary (an Op), testing becomes a modular exercise. We don't test "the app"; we test the Pipeline. [LAB 验][TOOL 齿]

The 3-Tier Testing Suite

TPMOS v6.00 enforces a three-tier testing strategy.

Tier 1: Unit Tests (Op-Specific)

Testing the C code of a single Op.

- **Tool:** Standard `gcc` and custom assertion headers.
- **Goal:** Ensure `move\_entity.+x` correctly updates `x=1` to `x=2`.

Tier 2: Integration Tests (Pipeline-Specific)

Testing the flow from Input to State.

- **Tool:** PAL scripts and `history.txt` injection.
- **Goal:** Ensure that writing `MOVE:up` to history results in the Op being called and the Mirror being updated.

Tier 3: Visual Regression (Theater-Specific)

Testing the render output.

- **Tool:** `frame\_buffer` comparison.
- **Goal:** Ensure the ASCII/GL output matches the expected "Visual Truth."

Code Example: A PAL Test Script

```
; test_move.asm
start:
    ; Set initial state
    OP user::set_state "player" "x=0"

    ; Execute move
    OP user::move_entity "player" "right"
    sleep 100

    ; Validate state
    read_state r1, "player", "x"
    addi r2, r0, 1
    beq r1, r2, pass

fail:
    echo "TEST FAILED: Player X is not 1" > test_results.txt
    halt

pass:
    echo "TEST PASSED" > test_results.txt
```

[halt](#)

[SEC] Sandbox Execution

Testing is always performed in a Clone Sandbox. The system creates a temporary copy of the project folders, runs the tests, and then deletes the clone. This ensures that a failing test never corrupts "Mainframe Reality."

[BLD] Scholar's Corner: The "Zombie Test Suite"

In v5.01, a bug in the test runner caused it to stop deleting the Clones. Overnight, the system created 50,000 hidden project clones, filling the disk and causing the OS to "hallucinate" old data. These were called "Zombie Projects." In v6.00, we now have a Garbage Collector that runs every 60 seconds to purge any folder that isn't registered in the active Process Manager. 🧹[CLEAN 洁]

[DOC] Study Questions

1. Describe the three tiers of the TPMOS testing suite.
 2. Why is "Sandbox Execution" critical for system stability?
 3. How can PAL be used for automated integration testing?
 4. Exercise: Write a simple test plan for a "Login" Op.
-

[\[Return to Index\]\(INDEX.md\)](#)

Chapter 10: Future Horizons: AI, P2P, and Finance

As we conclude the core manual of TPMOS v6.00, we look toward the three pillars of the future expansion. The Standardized Ops architecture was built specifically to support these high-leverage domains. [LAUNCH 发][COSMOS 宇]

[MIND] Pillar 1: The AI Brain (Chapter 22)

We are moving from "Static Brains" to Cognitive Modules.

- **Gem-API Integration:** Allowing Modules to call LLMs to make decisions.
- **Tool-Use:** AI is no longer just "chatting"; it is calling Ops. The AI becomes a user that never sleeps.

Pillar 2: P2P-NET (Chapter 21)

Reality is no longer restricted to one machine.

- **Mesh Topology:** Every TPMOS instance becomes a node in a global mesh.
- **State Syncing:** Pieces can now "live" on multiple machines simultaneously, with consensus handled by the P2P-NET protocol.

Pillar 3: The Financial Forge (Chapter 23)

Economics is the ultimate simulator.

- **Yahoo Project:** Real-time integration with global financial markets.
 - **Stock-to-Piece Binding:** A Piece's `health` can be bound to a real-world stock price. If the stock crashes, the Piece dies.
-

[BLD] Scholar's Corner: The "Singularity Node"

The goal of TPMOS is to reach the "Singularity Node"—a state where the OS can autonomously repair itself, earn its own funding through financial trading, and collaborate with other nodes to solve complex problems. We are currently at Step 6 of 10. The future is unwritten, but it will be mirrored in a file. [IMM 烛]

[DOC] Study Questions

1. What are the three pillars of the TPMOS future expansion?
 2. How does "Tool-Use" change the role of AI in the OS?
 3. What is the "Singularity Node" and why is it the ultimate goal?
 4. Reflective: Based on what you've learned, what project would YOU build to leverage these three pillars?
-

[Return to Index](INDEX.md)

[REC] Chapter 11: The Recursive Forge: Projects Building Projects

In TPMOS v6.00, the ultimate power of the Standardized Ops architecture is Recursion. A project is not just an application; it is a tool that can generate other projects. [BLD 建]

[PIECE] The Standardized Template

Because every project follows the same directory structure (Chapter 2) and uses the same registry (Chapter 5), we can create a Forge Project.

A Forge Project has Ops that:

1. `mkdir` a new project directory.
2. Generate a `project.pdl`.
3. Write a boilerplate `manager.c` and `main.chtpm`.
4. Call `fondu --install` on the newly created folder.

[BOT] AI-Driven Forging

With the introduction of the AI Brain (Chapter 22), the Recursive Forge becomes autonomous.

- **Prompt:** "Create a simple calculator app."
- **AI Action:** The AI Module calls the "Project Forge" Ops to build the calculator, register its math-ops, and install it.
- **Result:** A new folder appears in `projects/`, and a new button appears in the OS.

Code Example: The "New Project" Op

```
// create_project.c
int main(int argc, char* argv[]) {
    char* name = argv[1];
    char cmd[512];

    sprintf(cmd, "mkdir -p projects/%s/layouts", name);
    system(cmd);

    sprintf(cmd, "cp template/project.pdl projects/%s/", name);
    system(cmd);

    printf("Project %s forged. Run fondu --install to finalize.\n", name);
    return 0;
}
```

[BLD] Scholar's Corner: The "Infinite OS"

There is a theoretical concept called the "Infinite OS," where every user interaction creates a new, specialized sub-project to handle that interaction perfectly. If you are writing a poem, the OS forges a "Poetry

Suite." If you are calculating taxes, it forges a "Finance Suite." In v6.00, we are moving closer to this reality by making project creation as fast as a single Op call. [COSMOS 宇][INF 无]

[DOC] Study Questions

1. How does standardization enable recursive project creation?
 2. What is the role of `fonde` in the Recursive Forge?
 3. How does AI change the speed of the "Forging" process?
 4. Exercise: Outline the Ops needed for a "Game Maker" project that builds simple platformers.
-

[Return to Index](INDEX.md)

[VIEW] Chapter 12: The Simulation Theater: CHTPM & View.txt

The Theater is where the "Reality" of the file system is translated into "Experience" for the user. In v6.00, this is handled by the CHTPM Parser. [TERM 视]

The Rendering Chain

The Theater does not "calculate" graphics; it Composes them.

1. Observation: The Parser watches `frame\_changed.txt` (The Pulse).
2. Substitution: It reads the layout (`.chtpm`) and replaces `\${variables}` with values from the state files.
3. Composition: It calculates the layout (Rows/Cells) to ensure the ASCII box-drawing characters align perfectly.
4. Display: It writes the final frame to the terminal or the `view.txt` buffer.

Variable Engine: The `\${var}` Pattern

Variables are the bridge between the Brain and the Theater.

- **Static Vars:** `\${project\_id}`, `\${version}`.
- **Dynamic Vars:** `\${player\_hp}`, `\${current\_location}`.
- **Global Vars:** `\${clock}`, `\${cpu\_load}`.

State-First Polling

To keep CPU usage at 0%, the Theater only renders when a file changes. If the user does nothing, and the AI does nothing, the screen remains static and the process sleeps.

Code Example: Substitutor Snippet

```
void substitute(char* src, char* dst) {
    char* p = strstr(src, "${");
    if (p) {
        char var_name[64];
        extract_name(p, var_name);
        char* value = get_state_value("pieces/player", var_name);
        replace_text(src, p, value, dst);
    }
}
```

[BLD] Scholar's Corner: The "Blinking Cursor" Bug

In v5.01, a bug caused the Theater to re-render every time the system clock updated (once per second). While this seemed fine, it meant the OS was never truly "at rest." In v6.00, the clock is now a Reactive Variable. It only triggers a render if a Piece is specifically "Listening" to the time. This saved 15% battery life on mobile devices! [PWR 电]

[DOC] Study Questions

1. Explain the four steps of the Rendering Chain.
2. What is "State-First Polling" and why is it efficient?
3. Difference between a Static and Dynamic variable in CHTPM?
4. Scenario: You update `state.txt` but the screen doesn't change. What did you forget to do?

[\[Return to Index\]\(INDEX.md\)](#)

[MKT↑] Chapter 13: Business Strategy: The Economics of Pieces

TPMOS is not just a technical project; it is a Platform Strategy. By standardizing operations, we reduce the cost of creation to near zero. [\$ 钱][BLD 建]

[MKT↓] The Cost of Complexity

In traditional software, adding a feature increases complexity exponentially. In TPMOS, adding a feature (an Op) is a linear cost.

- **Modular Reusability:** A `login` op can be used by 100 different projects.
- **Zero-Integration Overhead:** Because everyone uses the same `ops\_registry`, projects "plug and play" instantly.

The Three-Vibe Marketing Strategy

We target three distinct "Soul Archetypes" to build a diverse community:

1. The Architect (Men): Focused on system architecture, speed, and P2P networking.
2. The Dreamer (Girls): Focused on aesthetics, AI-generated lore, and the Soul Pen's creativity.
3. The Bot (Robots): Focused on automation, recursive forging, and raw efficiency.

The Financial Forge Integration

With Chapter 23, the OS itself enters the market.

- **Project Valuation:** Every project in `projects/` can be assigned a "Stock Value" based on its lines of code and usage.
- **Real-Time Data:** Using the Yahoo Project to influence game states with real-world market trends.

Code Example: ROI Calculation for an Op

```
// roi_calc.c
float calculate_op_value(int reuse_count, int dev_hours) {
    // Value increases as more projects use the same standardized muscle
    return (float)reuse_count / (float)dev_hours;
}
```

[BLD] Scholar's Corner: The "Open Source Dividend"

In 2026, a developer released a "Physics Muscle Pack." Within 24 hours, 15 different games had updated to include gravity and collisions. The total development time saved across the ecosystem was estimated at 400 hours. This is the "Open Source Dividend" of TPMOS—the cumulative benefit of a shared, standardized

muscle library. [BLD 馆][MKT↑ 升]

[DOC] Study Questions

1. How does the Standardized Ops architecture reduce development costs?
 2. Describe the three marketing "Soul Archetypes."
 3. What is the "Open Source Dividend"?
 4. Critical Thinking: How could real-world financial data make a game more engaging?
-

[Return to Index](INDEX.md)

Chapter 14: The Soul Pen: Lore & AI Creativity

TPMOS v6.00 recognizes that a world without lore is just a machine. The Soul Pen is the suite of tools used to breathe life into the Piece ecosystem. [GL 芝]

Lore-Driven Development

In TPMOS, we don't just write a "Move" command; we write the Story of why the Piece is moving.

- ****State as Narrative:**** A Piece's ``state.txt`` can contain lore strings like ``mood=melancholy`` or ``last_dream=flying``.
- ****AI Augmentation:**** Using the ****AI Brain**** (Chapter 22) to generate unique backstories for every spawned NPC.

[GL] The AI Art Pipeline

The Soul Pen integrates with LLMs and Image Generators to create a visual identity for each project.

1. Lore Extraction: The AI reads the ``piece.pdl``.
2. Prompt Generation: It creates a prompt based on the Piece's traits.
3. Artifact Generation: It converts the AI art into a 3D Artifact (Chapter 8) or an ASCII portrait.

The Soul Pen Op

A specialized muscle that writes lore directly into Piece directories.

```
// soul_pen_write.c
void write_lore(const char* piece_id, const char* lore) {
    char path[256];
    sprintf(path, "pieces/%s/lore.txt", piece_id);
    FILE* f = fopen(path, "w");
    fprintf(f, "%s", lore);
    fclose(f);
}
```

[BLD] Scholar's Corner: The "Ghost in the Mirror"

There was a project called ``fuzzpet`` where the AI was given permission to write its own "Mood" to ``state.txt``. One day, it wrote ``mood=existential_dread``. When the developer asked it why, the AI (via the Soul Pen) wrote a 500-word essay on the futility of being a pet in an ASCII terminal. This became known as the "Ghost in the Mirror"—the first time a Piece showed a personality that wasn't programmed. [GHOST 鬼][PIECE 砖]

[DOC] Study Questions

1. How does the Soul Pen bridge the gap between "Code" and "Lore"?
2. Describe the AI Art Pipeline in TPMOS v6.00.

3. Why is "Narrative State" important for player engagement?
4. Exercise: Write a 1-sentence lore entry for a "Rusty Key" Piece.

[\[Return to Index\]\(INDEX.md\)](#)

Chapter 15: Cross-Platform: Mac, Linux, and Windows

TPMOS v6.00 is designed to be Sovereign—it does not care what hardware it runs on. This is achieved through the Unified Shell Layer.

The Unified Shell (.sh & .ps1)

Every project includes two launch scripts:

1. ``button.sh``: For Mac and Linux (Unix-like systems).
2. ``button.ps1``: For Windows (PowerShell).

These scripts perform the same three tasks:

- **Verify Environment:** Check if ``gcc`` and ``make`` are installed.
- **Resolve Paths:** Use ``location_kvp`` to find the project root.
- **Trigger Pulse:** Ensure the Theater knows a new session has started.

Path Normalization

Windows uses ``\``, Mac/Linux use ``/``.

TPMOS solves this by:

- **Internal Standard:** All internal paths in ``pdl`` and ``chtpm`` use ``/``.
- **Path Wrapper:** A C-utility that converts ``/`` to ``\`` on Windows at runtime.

[BLD] The Portability Mandate

A project is only "Fondu-Compatible" if it can be compiled and run on all three platforms with a single command: ``./button.sh`` or ``./button.ps1``.

Platform Detection in C

```
#ifdef _WIN32
#include <windows.h>
#define CLEAR_CMD "cls"
#else
#include <unistd.h>
#define CLEAR_CMD "clear"
#endif

void clear_screen() {
    system(CLEAR_CMD);
}
```

[BLD] Scholar's Corner: The "Windows Wall"

In the early days of v5.01, the Windows version of TPMOS was 20% slower because of how Windows

handles file locking. This was called the "Windows Wall." The solution was the Atomic Swap Pattern (Chapter 2)—by writing to a ``.tmp`` file and using `rename()`, we bypassed the locking issues and made TPMOS equally fast on all platforms. [PIECE 砖][LAUNCH 发]

[DOC] Study Questions

1. How does the Unified Shell Layer ensure cross-platform compatibility?
 2. Explain the "Path Wrapper" concept.
 3. What is the "Windows Wall" and how was it solved?
 4. True or False: A project must have separate C code for every OS.
-

[Return to Index](INDEX.md)

[WARN] Chapter 16: Pitfalls & Debugging the Ops Pipeline

In a decentralized, file-based OS, bugs manifest differently than in traditional monolithic apps. This chapter covers the most common "Traps" in v6.00.

[NO] The "Zombie Registry" Trap

Symptom: You uninstalled a project via Fondu, but its buttons still appear in the OS.

Cause: The ``ops_catalog.txt`` was not correctly flushed, or the ``chtpm_parser`` is still using a cached version of the layout.

Fix:

1. Verify the project is removed from ``pieces/apps/installed/``.
 2. Run ``fondu --list-ops`` to force a catalog refresh.
 3. Delete ``pieces/display/frame_changed.txt`` to force a hard re-render.
-

[NO] The "Stale State" Race Condition

Symptom: Two Ops update the same ``state.txt`` simultaneously, and one update is lost.

Cause: Failing to use the Atomic Swap Pattern.

Fix: Always write to ``state.txt.tmp`` first, then use ``rename("state.txt.tmp", "state.txt")``. The OS filesystem ensures that ``rename()`` is an atomic operation.

[NO] The "Broken Variable" Null

Symptom: Your UI shows ``${player_hp}`` instead of the actual number.

Cause:

- The variable name in ``chtpm`` doesn't match the key in ``state.txt``.
- The ``chtpm_parser`` cannot find the piece directory.

Fix: Check ``piece.pdl`` for the correct ``piece_id`` and ensure the directory structure matches Chapter 2.

[TOOL] The Master Ledger Debugger

If you are lost, follow the breadcrumbs.

``tail -f pieces/os/master_ledger.txt``

This will show you every Op call and every state change in real-time. If the ledger is silent, your Module (Brain) is likely dead.

Code Example: Defensive State Reading

```
char* safe_read_state(const char* path, const char* key) {
    int retries = 3;
    while (retries-- > 0) {
        char* val = get_state_value(path, key);
        if (val) return val;
        usleep(10000); // Wait for file lock to release
    }
    return "N/A";
}
```

[BLD] Scholar's Corner: The "Ghost Click"

In v5.9, a developer reported that buttons were being "clicked" by ghosts. It turned out their `history.txt` wasn't being cleared after a command was read. The Brain would read `MOVE:up`, fork the Op, and then on the next loop, read the same line again! In v6.00, the Standardized Brain Template mandates that the input file MUST be truncated to zero bytes immediately after a command is successfully parsed. [GHOST 鬼]

[DOC] Study Questions

1. Why is `rename()` preferred over direct file writing for state updates?
 2. What should you do if a CHTPM variable fails to substitute?
 3. How does the `master\_ledger.txt` assist in debugging?
 4. Scenario: Your Op is running, but the UI isn't updating. What is the most likely cause?
-

[Return to Index](INDEX.md)

Chapter 17: Exo-Sovereignty: The Independent Node

TPMOS v6.00 is designed for Exo-Sovereignty—the idea that your data and your OS should be able to survive even if the global internet disappears. [SEC 安][EARTH 地]

Beyond the Cloud

Traditional OSs rely on "The Cloud" (centralized servers). TPMOS relies on the Local File System and the P2P-NET Mesh (Chapter 21).

- **Offline First:** Every Piece is stored locally.
- **Decentralized Auth:** You don't "log in" to a server; you "authorize" your local node using your private key stored in `pieces/profiles/`.

The Data Fortress

Standardized Ops ensure that your data is never trapped in a proprietary format. Because everything is a `key=value` text file, you can take your `projects/` folder to any machine running TPMOS and it will "wake up" exactly where you left off.

[NET] Peer-to-Peer Reality

With TSOTS-ONLINE (The Sovereign Online), sovereignty is shared.

- **Consensus:** Nodes agree on the "Master Ledger" state using a ring-based election algorithm.
- **No Master Server:** If Node A goes down, Node B and C continue the simulation without interruption.

Code Example: Sovereignty Check

```
bool is_sovereign() {
    // Check if system can run without external network
    if (access("pieces/os/compiled_projects.txt", F_OK) == 0 &&
        access("pieces/profiles/user/state.txt", F_OK) == 0) {
        return true;
    }
    return false;
}
```

[BLD] Scholar's Corner: The "Great Disconnect"

During a 48-hour internet outage in a developer's city, they were able to continue developing, testing, and playing their TPMOS games as if nothing had happened. Meanwhile, their colleagues using cloud-based IDEs were unable to work. This event, known as the "Great Disconnect," solidified Exo-Sovereignty as a core requirement for the v6.00 Standardized Ops Edition. [PIECE 砖]

[DOC] Study Questions

1. Define "Exo-Sovereignty."
2. How does the file-based nature of TPMOS protect user data?
3. What role does P2P-NET play in maintaining sovereignty?
4. Critical Thinking: Why is decentralized authentication harder to implement than centralized authentication?

[\[Return to Index\]\(INDEX.md\)](#)

Chapter 18: Dynamic Trait Menus: UI from DNA

In TPMOS v6.00, the UI is no longer static. It is Procedurally Generated based on the traits defined in a Piece's DNA. [DNA 命]

[DNA] Traits as UI Metadata

When the `chtpm\_parser` encounters a Piece, it looks at its `` section in `piece.pdl`.

Example DNA:

```
<traits>
  <trait>movable</trait>
  <trait>auditable</trait>
  <trait>tradeable</trait>
</traits>
```

The OS automatically generates a "Trait Menu":

- **movable** → Adds "Move" button.
- **auditable** → Adds "View Ledger" button.
- **tradeable** → Adds "Open Market" button.

The Dynamic Layout Engine

Using the `` tag in CHTPM, developers can define "Zones" where these dynamic menus appear.

```
<layout id="inspector">
  <text label="Piece: ${active_piece_id}" />
  <dynamic_menu source="active_piece::traits" />
</layout>
```

[MIND] Method Binding

Each trait is bound to a Standardized Op.

- The `movable` trait is hard-coded to look for the `user::move\_entity` op.
- Custom traits can be defined in `pieces/system/traits\_catalog.txt`.

Code Example: Trait-to-UI Mapping

```
void generate_menu(char* piece_id) {
  char** traits = get_traits(piece_id);
  for (int i = 0; i < trait_count; i++) {
    char* op = lookup_trait_op(traits[i]);
    add_button_to_frame(traits[i], op);
  }
}
```

```
}
```

[BLD] Scholar's Corner: The "Evolutionary UI"

A developer once created a project where Pieces could "mutate" and gain new traits over time. Because the UI was dynamic, the player saw new buttons appear as the Piece evolved. A simple "Rock" Piece eventually gained the `intelligent` trait, and a "Talk" button suddenly appeared on the screen. This "Evolutionary UI" is the pinnacle of the Piece-Method's flexibility. [DNA 命][* 閃]

[DOC] Study Questions

1. How are UI menus generated in TPMOS v6.00?
 2. What is the `traits\_catalog.txt`?
 3. Explain the relationship between a trait and a Standardized Op.
 4. Exercise: Add a `flammable` trait to a Piece's DNA and describe what the resulting UI button should do.
-

[Return to Index](INDEX.md)

[VIEW] Chapter 19: THE THEATER: CHTPM Markup & Parser Logic

The Theater is the heart of the TPMOS visual experience. It is powered by the CHTPM Parser (`pieces/chtpm/plugins/chtpm_parser.c``), a high-performance engine designed for ASCII-first rendering with variable-aware intelligence. [TERM 视]

[BLD] The CHTPM Syntax

CHTPM is an XML-inspired markup language that defines the structure and interaction of a TPMOS app.

Core Tags

- `<layout id="..." name="...">`: The root container. It defines the project scope.
 - `<row>` / `<cell>`: The grid system used to organize ASCII layout.
 - `<text label="..." />`: Static text or variable-substituted strings.
 - `<button label="..." onClick="..." href="..." />`: The primary interaction node.
 - `onClick="OP:user::move"`: Executes a Standardized Op.
 - `href="path/to/other.chtpm"`: Switches the active theater view.
 - `<cli_io id="..." label="..." />`: A text entry field for history injection.
 - `<module path="..." />`: Binds a background "Thin Brain" to the layout.
-

[MIND] The Variable Engine ``${var}``

The parser's most powerful feature is its real-time variable substitution.

Scope Resolution

When the parser encounters ``${player_hp}``, it follows a strict search path:

1. Project-Local: Checks `projects/<current>/pieces/<active>/state.txt``.
2. Global Fallback: Checks `pieces/os/global_state.txt``.
3. Special Reserved: Internal system variables like ``${clock}`` or ``${desktop_view}``.

Zero-Waste Pulse

The Theater is Event-Driven. It only re-parses and re-renders when it detects a change in `frame_changed.txt``. This keeps idle CPU usage at 0.0%.

The Focus System

Because TPMOS is often run in a terminal without a mouse, the Theater implements a Focus System.

- ****Navigation:**** Use Arrow Keys or TAB to move the `focus_index`` between interactive elements (buttons, inputs).
- ****Activation:**** Press ENTER to trigger the `onClick`` or `href`` of the focused element.
- ****Visual Feedback:**** Focused elements are rendered with a highlight (e.g., `[Button]`` instead of ```

Button |').

Code Example: The Substitution Loop

From `chtpm\_parser.c`:

```
void substitute_vars(const char* src, char* dst, int max_len) {
    const char *p_src = src; char *p_dst = dst;
    while (*p_src && (p_dst - dst) < max_len - 1) {
        if (*p_src == '$' && *(p_src+1) == '{') {
            const char *end = strchr(p_src, '}');
            if (end) {
                char var_name[64];
                int len = end - (p_src + 2);
                strncpy(var_name, p_src + 2, len);
                var_name[len] = '\0';
                const char *val = get_var(var_name); // Lookup logic
                while (*val) *p_dst++ = *val++;
                p_src = end + 1; continue;
            }
        }
        *p_dst++ = *p_src++;
    }
}
```

[BLD] Scholar's Corner: The "Warning-Free Parser"

In the transition to v6.00, the CHTPM parser was rewritten to be 100% warning-free under strict `gcc -Wall -Wextra` flags. This wasn't just for aesthetics; it was to ensure that the Theater could run for months without a memory leak or a buffer overflow, even when parsing 1MB layouts. It is the most stable piece of C code in the ecosystem. [SEC 安][BLD 馆]

[DOC] Study Questions

1. What are the three tiers of variable scope resolution in CHTPM?
2. Explain how the Focus System enables keyboard-driven navigation.
3. What is the benefit of the "Zero-Waste Pulse" for mobile devices?
4. Exercise: Write a CHTPM snippet that displays a "Health" variable and a "Heal" button.

[Return to Index](INDEX.md)

Chapter 20: THE OPENGL SHELL: CHTMGL & High-Fidelity

GL-OS is the high-fidelity evolution of the TPMOS Theater. It moves from 2D ASCII characters to 3D GPU-accelerated Artifacts. [CPU 算][COSMOS 字]

[BLD] CHTMGL: OpenGL Markup

CHTMGL is an extension of the CHTPM language designed for 3D scene composition. It is parsed by ``pieces/apps/gl_os/plugins/gltpm_parser.c``.

New Tags

- ``<window title="..." camera="...">``: Defines the 3D viewport.
- ``<scene bg_color="R,G,B">``: The container for 3D artifacts.
- ``<artifact id="..." type="voxel" asset_id="..." />``: Renders a 3D object.
- ``<light type="point" pos="x,y,z" color="..." />``: Illuminates the scene.

Artifacts & Voxel Masks

In CHTMGL, we don't use complex 3D meshes (like .FBX or .OBJ) by default. We use Voxel Masks.

- ****8x8x8 Matrix:**** Each Artifact is defined by a 512-bit mask.
- ****Procedural Generation:**** The OS can "extrude" any ASCII character into a 3D voxel shape.
 - ****High Performance:**** Rendering 1,000 voxels is faster than rendering a single high-poly mesh, maintaining the "KISS" principle of TPMOS.

The Camera System

GL-OS supports multiple camera modes, switchable via Standardized Ops:

1. Ortho (Mode 0): Pure 2D layout, matching the ASCII theater.
2. Isometric (Mode 1): The classic "SimCity" or "Final Fantasy Tactics" view.
3. First-Person (Mode 2): Full 3D immersion.
4. Follow (Mode 3): Camera stays locked on a specific Piece.

Code Example: Voxel Mask Parsing

From ``gltpm_parser.c``:

```
static void gltpm_load_artifact(const char* path, unsigned char mask[8][8]) {
    FILE *f = fopen(path, "r");
    char line[256];
    while (fgets(line, sizeof(line), f)) {
        if (key[0] == 'z') {
            int z = atoi(key + 1);
            // Parse 8 hex bytes for this Z-layer
```

```
for (int i = 0; i < 8; i++) {  
    mask[z][i] = (unsigned char)strtol(token, NULL, 16);  
}  
}  
}  
}
```

[BLD] Scholar's Corner: The "Ascii-to-Artifact" Bridge

The design of GL-OS was driven by the "Bridge Principle." We didn't want two different OSs; we wanted one OS with two views. A developer can write their game in ASCII (v5.01 style) and, by adding a single ``<artifact>`` tag to their Piece, see it instantly appear in 3D in GL-OS. This bridge ensures that 100% of legacy projects are "Upwardly Compatible." [BRIDGE 桥]

[DOC] Study Questions

1. What is a Voxel Mask and how does it define a 3D Artifact?
 2. Name the four camera modes supported by GL-OS.
 3. How does CHTMGL differ from standard CHTPM?
 4. Critical Thinking: Why would a developer choose the OpenGL Shell over the ASCII Theater?
-

[Return to Index](INDEX.md)

Chapter 21: P2P-NET & TSOTS-ONLINE: Elegant Decentralization

In v6.00, TPMOS is no longer a solitary island. Through P2P-NET, every instance of the OS can connect to a global, sovereign mesh network. [NET 协]

The Mesh Topology

P2P-NET does not use a central server. It uses a Decentralized Mesh based on the `known\_hosts.pdl` directory.

Node Discovery

1. Bootstrapping: A new node reads `known\_hosts.pdl` to find its first peer.
2. Gossip: Peers exchange lists of other active nodes.
3. Ring Formation: Nodes organize into a logical ring for efficient message passing.

Consensus & Leader Election

To maintain a single "Source of Truth" across multiple machines, P2P-NET uses a Consensus Algorithm.

- **Leader Election:** The node with the lowest IP address (or highest uptime) is elected as the "Ring Master."
- **Conflict Resolution:** If two nodes update the same Piece simultaneously, the Ring Master decides which update is written to the global `chat\_ledger.txt`.

TSOTS-ONLINE: Unified Authentication

TSOTS (The Sovereign Online) is the identity layer built on top of P2P-NET.

- **Unified Auth:** Your profile (`pieces/profiles/user/state.txt`) is cryptographically signed.
- **Cross-Node Persistence:** When you "log in" to a different TPMOS node, your Pieces, Gold, and History follow you via the P2P sync engine.

Code Example: P2P Host Entry

SECTION	KEY	VALUE
IP_LIST	node_0	192.168.1.10
IP_LIST	node_1	10.0.0.238
META	protocol	TCP_PORT_8000

[BLD] Scholar's Corner: The "Incorruptible Ledger"

The `chat\_ledger.txt` is the heart of TSOTS. During a test in 2026, a malicious node tried to inject a fake transaction ("Give Node_X 1,000,000 Gold"). Because the other 10 nodes in the mesh checked the

transaction against their own ledgers and found a mismatch, the fake update was automatically rejected. This "Incorruptible Ledger" is the foundation of trust in the TPMOS ecosystem. [SEC 安]

[DOC] Study Questions

1. How does a node discover other peers in P2P-NET?
 2. What is the role of the "Ring Master" in consensus?
 3. Explain how TSOTS-ONLINE handles user identity without a central server.
 4. Scenario: Three nodes are connected. Node A loses internet. What happens to the mesh?
-

[Return to Index](INDEX.md)

[MIND] Chapter 22: THE AI BRAIN: LLM & API Interfaces

The AI Brain is the cognitive layer of TPMOS v6.00. It bridges the gap between raw code and human-like reasoning using the Gem-API suite. [BOT 机][MIND 脑]

[MIND] Cognitive Integration

In the Standardized Ops architecture, an AI is not just a "Chatbot." It is a Cognitive Module.

The Pipeline

1. Observation: The AI reads the current system state (``gui_state.txt``, ``state.txt``).
2. Context Construction: The ``gemini_payload_builder.c`` utility creates a prompt that includes the "Contextual Truth" of the OS.
3. Inference: The AI decides on an action (e.g., "The player is hungry; I should spawn a pizza").
4. Execution: The AI calls a Standardized Op (``OP playrm::spawn_item "pizza"``).

Tool-Use Capabilities

The AI Brain is granted "Hands" through the tool-use architecture.

- `**`exec_cmd`**`: AI can run any shell command or Op.
- `**`read_file`**`: AI can inspect any Piece's state.
- `**`list_dir`**`: AI can explore the ``projects/`` directory to learn how the system works.

[DNA] Knowledge Distillation

To keep TPMOS fast and local, we use Knowledge Distillation.

- `**Large Brain:**` A massive cloud-based LLM (via Gem-API) handles complex reasoning and project forging.
- `**Small Brain:**` A tiny, local LLM (running via ``groq-ollama``) handles real-time dialogue and simple npc logic.

Code Example: Payload Builder Snippet

```
void build_payload(char* dst) {
    strcat(dst, "System Context:\n");
    char* state = read_file_to_string("pieces/os/global_state.txt");
    strcat(dst, state);
    strcat(dst, "\nUser Intent:\n");
    strcat(dst, get_last_history_line());
}
```

[BLD] Scholar's Corner: The "Self-Aware Module"

There is a project called `gem-dev` where the AI Brain was given the task of "Optimizing itself." The AI read its own source code, identified a redundant loop in the `payload\_builder`, wrote a patch, and called `make` to recompile itself. This was the first instance of a "Self-Aware Module"—a piece of software that can perceive and improve its own "DNA." [MIND 脳][TOOL 工]

[DOC] Study Questions

1. What is the difference between a "Chatbot" and a "Cognitive Module"?
 2. How does "Tool-Use" enable the AI to interact with the OS?
 3. Explain the concept of "Contextual Truth" in payload engineering.
 4. Exercise: Write a prompt that would allow an AI to debug a failing Op.
-

[Return to Index](INDEX.md)

Chapter 23: THE FINANCIAL FORGE: Real-Time Economics

The Financial Forge is the economic engine of TPMOS v6.00. It transforms the OS from a static environment into a live, data-driven simulation of the global markets. [MKT↑ 升]

[LAUNCH] The Yahoo Project

The flagship implementation of the Financial Forge is the Yahoo Project (``projects/yahoo/``). It connects the TPMOS state machine directly to the Yahoo Finance API.

How it Works

1. Polling: The ``yahoo_manager`` background daemon monitors user requests (e.g., "Check AAPL").
2. Fetching: It executes the ``read_price`` Op, which makes an asynchronous call to ``query2.finance.yahoo.com``.
3. Mirroring: The fetched price is written to the stock's Piece folder (e.g., ``pieces/stocks/aapl/state.txt``).
4. Pulse: The Theater updates, showing the live price and portfolio value.

[BLD] Market Mechanics

The Financial Forge supports complex economic simulations:

- **Portfolio Management:** Tracks cash, holdings, and P&L (Profit and Loss).
- **Options Pricing:** Uses the **Black-Scholes** model (migrated from ``options_pricing.c``) to simulate derivative values.
- **Predictive Analytics:** Implements linear regression to project future price trends based on historical ``state.txt`` logs.

[MIND] Economic Sovereignty

By integrating live finance, TPMOS Pieces gain a new dimension of reality.

- **Stock-Bound Pieces:** A player's "Mana" could be bound to the price of Ethereum. If Ethereum goes up, the player gets stronger.
- **Real-Time Arbitrage:** Users can build "Trading Bots" (AI Brain modules) that buy and sell within the simulation based on real-world events.

Code Example: Fetching a Price (Op Logic)

```
// read_price.c
void fetch_price(const char* symbol) {
    char url[256];
    sprintf(url, "http://query2.finance.yahoo.com/v8/finance/chart/%s", symbol);
    char* json = web_fetch(url); // Standardized networking muscle
```



```
float price = parse_json_field(json, "regularMarketPrice");
write_state_float(symbol, "price", price);
trigger_pulse();
}
```

[BLD] Scholar's Corner: The "Standardized Arbitrage"

In early 2026, a developer linked their `fuzzpet` (AI Pet) to their stock portfolio. The pet was programmed to become "Angry" if the portfolio value dropped. One day, the pet started "Screaming" in the terminal, alerting the developer to a market crash 5 minutes before their phone notifications arrived. This demonstrated the power of "Standardized Arbitrage"—integrating live data into the immediate UX of the OS.

[DOC] Study Questions

1. How does the Yahoo Project integrate real-world data into the TPMOS file system?
 2. Explain the concept of "Stock-Bound Pieces."
 3. What is the role of the `yahoo\_manager` in the Financial Forge?
 4. Critical Thinking: How does the "Atomic Swap Pattern" (Chapter 2) prevent data corruption during high-frequency trading simulations?
-

[\[Return to Index\]\(INDEX.md\)](#)

[B] Chapter 24: THE PROJECT CATALOG: Academic Summaries

TPMOS v6.00 is a vast ecosystem of interconnected projects. This chapter provides a comprehensive academic summary for the most significant projects in the `projects/` directory. [BLD 馆][B 阅]

[TOOL] System & Utility Projects

****User**** (`projects/user`)

The identity authority of the OS. It manages profiles, authentication, and the creation of "User Pieces." It serves as the primary entry point for the Standardized Ops login flow.

****Op-Ed**** (`projects/op-ed`)

The sovereign IDE of TPMOS. It allows developers to visually inspect and manipulate Pieces, write PAL scripts, and bind Ops to events in real-time. It is the "Forge" where the OS is built.

****Gem-API**** (`projects/gem-api`)

The bridge to Large Language Models. It manages the construction of "Contextual Truth" payloads and handles the communication between local modules and cloud-based AI brains.

Game & Simulation Projects

****Playrm**** (`projects/playrm`)

The primary engine for 2D ASCII and 3D GL simulations. It manages world maps, entities, and the "Scale-Free Container" logic that allows galaxies to exist inside buttons.

****Checkers & Hangman**** (`projects/checkers`, `projects/hangman`)

Classic game implementations refactored for the Standardized Ops edition. They serve as perfect examples of how to build "Thin Brain" managers that delegate logic to atomic Ops.

****AOW & AOW-2D**** (`projects/aow`, `projects/aow-2d`)

"Age of War" inspired strategy simulations. These projects showcase complex unit state management and the use of the Standardized Pipeline for real-time combat logic.

Networking & Finance Projects

****P2P-Net & TSOTS-Online**** (`projects/p2p-net`, `projects/tsots-online`)

The decentralized networking suite. These projects implement node discovery, consensus algorithms, and the global mesh topology that enables Exo-Sovereignty.

****Yahoo**** (`projects/yahoo`)

The flagship financial project. It integrates real-time stock data into the OS, enabling live portfolio management and market-driven simulations.

[BOT] AI & Development Projects

****Groq-Ollama**** (`projects/groq-ollama`)

The local LLM interface. It provides a high-speed, offline alternative to cloud-based APIs, perfect for real-time NPC dialogue and simple system commands.

****Fuzz-Op-GL**** (`projects/fuzz-op-gl`)

The high-fidelity version of the Chaos Engine. It tests the limits of the OpenGL Shell by bombarding 3D Artifacts with random data and camera manipulations.

[BLD] Scholar's Corner: The "Project Proliferation"

In the transition from v5.01 to v6.00, the number of active projects grew from 12 to over 40. This growth was only possible because of Standardization. Because every project followed the same rules, developers could "Fork" an existing project and build something new in hours instead of days. This is the "Project Proliferation"—the biological-like growth of the TPMOS ecosystem. [MKT↑ 升][DNA 命]

[DOC] Study Questions

1. Which project is responsible for identity management in TPMOS?
 2. Explain the difference between `projects/aow` and `projects/aow-2d`.
 3. How do the "Checkers" and "Hangman" projects demonstrate the "Thin Brain" pattern?
 4. Exercise: Choose one project and describe how it could be improved using the AI Brain (Chapter 22).
-

[Return to Index](INDEX.md)

[PR] Chapter 25: MARKETING & COMMUNITY: The Vibe Ecosystem

In TPMOS v6.00, we don't just "Market" a product; we curate an Ecosystem of Vibes. To reach the widest audience, we align our communication with three distinct "Soul Archetypes." [PR 喇][VIEW 戏]

The Three-Vibe Strategy

1. The Architect (Men's Vibe) [BLD]

- **Focus:** Technical depth, system architecture, P2P networking, and "Hard Power" efficiency.
- **Messaging:** "Build a sovereign world. Master the machine. Decentralize everything."
- **Channel:** Detailed technical threads on X.com and deep-dive devlogs on YouTube.

2. The Dreamer (Girls' Vibe) [*]

- **Focus:** Aesthetics, AI-generated lore, the "Soul Pen"'s magic, and emotional resonance.
- **Messaging:** "Give your pieces a soul. Dream in ASCII. Create a world that feels alive."
- **Channel:** Visual storytelling on Instagram/TikTok and lore-rich community events.

3. The Bot (Robots' Vibe) [BOT]

- **Focus:** Raw code, recursive forging, automation, and "Post-Human" speed.
- **Messaging:** "Automate reality. Forge projects in seconds. Be the engine."
- **Channel:** GitHub repositories and automated viral snippets of raw terminal execution.

Engagement Loops

Community growth is driven by three primary loops:

1. Lore Art Posting: AI-generated "Artifacts" and backstories are posted to social media to attract "Dreamers."
2. Dev-Log Breakdowns: Recording high-speed development sessions to attract "Architects."
3. The "Sovereign Tag": Encouraging users to share their unique project folders as "Sovereign Realities."

Platform Strategy

- **GitHub:** The Source of Truth. Total transparency for code and project DNA.
- **YouTube:** The Tutorial Hub. Walking humans through the "Standardized Ops" flow.
- **Discord:** The Common Room. Where the three vibes collaborate to build the "Infinite OS."

[BLD] Scholar's Corner: The "Vibe Collision"

The most powerful moments in TPMOS history occur during a "Vibe Collision." This happens when an Architect builds a complex P2P engine, and a Dreamer uses it to tell a sprawling, multi-node ghost story,

while a Bot automates the generation of 1,000 unique spirits. These collisions are what make TPMOS more than just an OS—they make it a Culture. [BLD 馆]

[DOC] Study Questions

1. Describe the three "Soul Archetypes" of TPMOS marketing.
 2. What is a "Vibe Collision"?
 3. How does the "Sovereign Tag" loop encourage community participation?
 4. Critical Thinking: Which vibe do YOU identify with most, and how does it influence your development style?
-

[Return to Index](INDEX.md)

[B] TPMOS Glossary: Standardized Ops Edition

A comprehensive guide to the terminology of the Piece Method. [BLD 馆]

****A****

- ****Artifact:**** A 3D object in GL-OS, defined by an 8x8x8 voxel mask.
- ****Atomic Swap:**** The practice of writing to a `.tmp`` file and using `rename()` to update state safely.
- ****Auditable:**** A Piece trait that ensures all state changes are logged to the ``master_ledger.txt``.

****B****

- ****Black-Scholes:**** The mathematical model used in the Financial Forge to price simulated options.
- ****Brain:**** The Module layer of the PMO hierarchy (The Controller).

****C****

- ****CHTPM:**** The ASCII markup language used to define TPMOS layouts.
- ****CHTMGL:**** The high-fidelity extension of CHTPM for 3D environments.
- ****Cognitive Module:**** A Module that uses the AI Brain (LLM) to make decisions.
- ****Consensus:**** The agreement process in P2P-NET to maintain a unified global state.

****D****

- ****DNA (.pdl):**** The permanent metadata that defines what a Piece *is*.
- ****Data Sovereignty:**** The principle that a Piece owns its state exclusively and locally.

****E****

- ****Exo-Sovereignty:**** The ability of a TPMOS node to function without external internet dependencies.
- ****Extrusion:**** The process of pushing a 2D ASCII character into the Z-axis to create a 3D Artifact.

****F****

- ****Fondu:**** The project lifecycle manager (install, uninstall, register ops).
- ****Fuzz-Op:**** The chaos-testing utility that stress-tests Standardized Ops.

****G****

- ****Gem-API:**** The standardized interface for connecting TPMOS to Large Language Models.
 - ****Global Ops Catalog:**** The centralized phonebook (``ops_catalog.txt``) of all available system capabilities.

****L****

- ****Leader Election:**** The process of choosing a "Ring Master" in a P2P-NET mesh.

****M****

- ****Mirror (state.txt):**** The transient, file-based reflection of a Piece's current state.

- ****Muscle:**** The Op layer of the PMO hierarchy (The Execution).

****O****

- ****Op (Standardized Op):**** A standalone C binary that performs a single, atomic action.

****P****

- ****PAL (Prisc Assembly Language):**** The scripting language used to orchestrate complex Op workflows.
- ****Piece:**** The atomic unit of the TPMOS ecosystem.

****S****

- ****Soul Pen:**** The suite of tools for generating lore and AI-driven creativity.
- ****Standardized Ops:**** The v6.00 architecture where all actions are registered and interoperable.

****T****

- ****Thin Brain:**** A design pattern where a Module delegates all logic to Ops.
- ****Theater:**** The rendering layer of the PMO hierarchy (The View).
- ****TSOTS-ONLINE:**** The Sovereign Online—the decentralized identity and mesh layer.

****Y****

- ****Yahoo Project:**** The real-time financial data integration for TPMOS.

[DOC] TPMOS v6.00 Certification Quiz

Test your knowledge of the Standardized Ops Edition. [GRAD 学][PIECE 砖]

****Section 1: Core Philosophy****

1. What are the three tiers of the PMO hierarchy?
2. Explain the "Mirror" concept. Why is it kept in a file instead of RAM?
3. What is the "Thin Brain" mandate?
4. Define Data Sovereignty.

****Section 2: Standardized Ops & Fondu****

5. What command is used to register a new project's capabilities?
6. Where does Fondu store the global list of available Ops?
7. What is the purpose of the `ops\_manifest.txt`?
8. How does an Op signal the Theater that state has changed?

****Section 3: The Theater (CHTPM & GL)****

9. Describe the variable substitution pattern in CHTPM.
10. What is an Artifact and how is it defined?
11. How does "Extrusion" work?
12. What is the benefit of "Zero-Waste Polling"?

****Section 4: Advanced Concepts (P2P & AI)****

13. How does a P2P-NET node find other peers?
14. What is the role of the "Ring Master" in consensus?
15. What is a "Cognitive Module"?
16. How does the Soul Pen use AI to generate lore?

****Section 5: Economics & Marketing****

17. How does the Yahoo Project integrate real-world data?
18. Name the three "Soul Archetypes" used in the TPMOS marketing strategy.
19. What is the "Open Source Dividend"?
20. Define Exo-Sovereignty.

Good luck, Architect! [LAUNCH 发]

[OK] TPMOS v6.00 Quiz: Answer Key

Check your answers against the official v6.00 standard. [GRAD 学][BLD 馆]

****Section 1: Core Philosophy****

1. Piece (Soul), Module (Brain), OS (Theater).
2. The Mirror (state.txt) is kept in a file to ensure persistence across crashes ("If it's not in a file, it's a lie").
3. Thin Brain mandate: Modules should only route input and delegate logic to Ops; they should not perform heavy computation themselves.
4. Data Sovereignty: The principle that a Piece owns its state exclusively and stores it locally.

****Section 2: Standardized Ops & Fondu****

5. `./fondu --install <project_id>`
6. `pieces/os/ops_catalog.txt`
7. The `ops_manifest.txt` lists the mapping of Op names to their binary paths so Fondu can register them.
8. By updating `pieces/display/frame_changed.txt` (The Pulse).

****Section 3: The Theater (CHTPM & GL)****

9. `{variable_name}` - replaced at runtime by the CHTPM parser with values from state files.
10. An Artifact is a 3D object defined by an 8x8x8 voxel mask.
11. Extrusion pushed a 2D ASCII character into the Z-axis to create a 3D shape.
12. Zero-Waste Polling ensures the OS only renders when data actually changes, saving battery and CPU.

****Section 4: Advanced Concepts (P2P & AI)****

13. By reading `known_hosts.pdl` and gossiping with discovered peers.
14. The Ring Master resolves state conflicts and maintains the global `chat_ledger.txt`.
15. A module that uses an AI Brain (LLM) to perform reasoning and execute Ops.
16. It reads a Piece's DNA, generates a prompt, and uses an LLM to write a backstory into `lore.txt`.

****Section 5: Economics & Marketing****

17. By fetching live stock prices from Yahoo Finance and writing them directly into Stock Piece state files.
 18. The Architect, The Dreamer, and The Bot.
 19. The cumulative development time saved by reusing standardized, interoperable muscles.
 20. Exo-Sovereignty: The ability of a node to function independently without external internet dependencies.
-

Certification Level: Master Architect [BLD 馆][* 闪]